

HDOC Day-7 Activity

Question 1:

Write a C program to find distance travelled in the nth second when initial velocity, acceleration, and n are given.

Step 1: Understanding the Problem

- **Input:** Initial velocity u (m/s), acceleration a (m/s²), and second number n .
- **Output:** Distance travelled in the nth second, s_n (m).
- **Formula:** $s_n = u + (a/2)(2n - 1)$

Step 2: Standard Test Case Table

Test Case	Input	Expected Output	Actual Output	Result
1	u=0, a=2, n=3	5.00 m	5.00 m	Pass
2	u=10, a=2, n=4	17.00 m	17.00 m	Pass
3	u=5, a=4, n=2	11.00 m	11.00 m	Pass

Step 3: Algorithm and Evaluation

1. Start.
2. Declare u , a , n and distance.
3. Read u , a and n from the user.
4. Calculate distance = $u + (a / 2) * (2 * n - 1)$.
5. Display the distance travelled in the nth second.
6. Stop.

Evaluation: The formula for displacement in the nth second is applied directly. The calculated values match the expected outputs for all prepared test cases.

Step 4: C Programming

```
#include <stdio.h>

int main()
{
    float u, a, distance;
    int n;

    printf("Enter initial velocity (m/s): ");
    scanf("%f", &u);
    printf("Enter acceleration (m/s^2): ");
    scanf("%f", &a);
    printf("Enter n: ");
    scanf("%d", &n);

    distance = u + (a / 2) * (2 * n - 1);
    printf("Distance in nth second = %.2f m\n", distance);
    return 0;
}
```

Step 5: Compilation and Execution

The program was compiled and executed in Kali Linux and evaluated against all three test cases.

Step 6: Kali Linux Compilation and Execution

```
(kali@kali)-[~]  
$ nano nth_distance.c  
(kali@kali)-[~]  
$ gcc nth_distance.c -o nth_distance
```

Test Case 1:

```
(kali@kali)-[~]  
$ ./nth_distance  
Enter initial velocity (m/s): 0  
Enter acceleration (m/s^2): 2  
Enter n: 3  
Distance in nth second = 5.00 m
```

Test Case 2:

```
(kali@kali)-[~]  
$ ./nth_distance  
Enter initial velocity (m/s): 10  
Enter acceleration (m/s^2): 2  
Enter n: 4  
Distance in nth second = 17.00 m
```

Test Case 3:

```
(kali@kali)-[~]  
$ ./nth_distance  
Enter initial velocity (m/s): 5  
Enter acceleration (m/s^2): 4  
Enter n: 2  
Distance in nth second = 11.00 m
```

Question 2:

Write a C program to find acceleration due to gravity when mass and radius of a planet are given.

Step 1: Understanding the Problem

- **Input:** Mass M of the planet (kg) and radius R (m).
- **Output:** Acceleration due to gravity g (m/s²).
- **Formula:** $g = GM / R^2$, where $G = 6.67430 \times 10^{-11} \text{ N}\cdot\text{m}^2/\text{kg}^2$

Step 2: Standard Test Case Table

Test Case	Input	Expected Output	Actual Output	Result
1	M=5.972e24, R=6.371e6	9.82 m/s ²	9.82 m/s ²	Pass
2	M=6.39e23, R=3.3895e6	3.71 m/s ²	3.71 m/s ²	Pass
3	M=1.898e27, R=6.9911e7	25.92 m/s ²	25.92 m/s ²	Pass

Step 3: Algorithm and Evaluation

1. Start.
2. Declare mass, radius, gravity and constant G.
3. Read mass and radius from the user.
4. Calculate gravity = (G * mass) / (radius * radius).
5. Display acceleration due to gravity.
6. Stop.

Evaluation: Newton's gravitational relation is used to calculate surface gravity from the given mass and radius. The outputs agree with the expected test-case values.

Step 4: C Programming

```
#include <stdio.h>

int main()
{
    double mass, radius, gravity;
    const double G = 6.67430e-11;

    printf("Enter mass of planet (kg): ");
    scanf("%lf", &mass);
    printf("Enter radius of planet (m): ");
    scanf("%lf", &radius);

    gravity = (G * mass) / (radius * radius);
    printf("Acceleration due to gravity = %.2f m/s^2\n", gravity);
    return 0;
}
```

Step 5: Compilation and Execution

The program was compiled and executed in Kali Linux and evaluated against all three test cases.

Step 6: Kali Linux Compilation and Execution

```
(kali@kali)-[~]  
$ nano gravity.c  
(kali@kali)-[~]  
$ gcc gravity.c -o gravity
```

Test Case 1:

```
(kali@kali)-[~]  
$ ./gravity  
Enter mass of planet (kg): 5.972e24  
Enter radius of planet (m): 6.371e6  
Acceleration due to gravity = 9.82 m/s^2
```

Test Case 2:

```
(kali@kali)-[~]  
$ ./gravity  
Enter mass of planet (kg): 6.39e23  
Enter radius of planet (m): 3.3895e6  
Acceleration due to gravity = 3.71 m/s^2
```

Test Case 3:

```
(kali@kali)-[~]  
$ ./gravity  
Enter mass of planet (kg): 1.898e27  
Enter radius of planet (m): 6.9911e7  
Acceleration due to gravity = 25.92 m/s^2
```

Question 3:

Write a C program to find parallel resistance when two resistances are given.

Step 1: Understanding the Problem

- **Input:** Two resistances R1 and R2 in ohms (Ω).
- **Output:** Equivalent parallel resistance Rp in ohms (Ω).
- **Formula:** $R_p = (R1 \times R2) / (R1 + R2)$

Step 2: Standard Test Case Table

Test Case	Input	Expected Output	Actual Output	Result
1	R1=10 Ω , R2=20 Ω	6.67 Ω	6.67 Ω	Pass
2	R1=100 Ω , R2=100 Ω	50.00 Ω	50.00 Ω	Pass
3	R1=4 Ω , R2=6 Ω	2.40 Ω	2.40 Ω	Pass

Step 3: Algorithm and Evaluation

1. Start.
2. Declare R1, R2 and parallel_resistance.
3. Read R1 and R2 from the user.
4. Calculate $\text{parallel_resistance} = (R1 * R2) / (R1 + R2)$.
5. Display the equivalent parallel resistance.
6. Stop.

Evaluation: The standard two-resistor parallel formula is applied directly. The calculated resistance matches the expected output in each test case.

Step 4: C Programming

```
#include <stdio.h>

int main()
{
    float r1, r2, parallel_resistance;

    printf("Enter resistance R1 (ohm): ");
    scanf("%f", &r1);
    printf("Enter resistance R2 (ohm): ");
    scanf("%f", &r2);

    parallel_resistance = (r1 * r2) / (r1 + r2);
    printf("Parallel resistance = %.2f ohm\n", parallel_resistance);
    return 0;
}
```

Step 5: Compilation and Execution

The program was compiled and executed in Kali Linux and evaluated against all three test cases.

Step 6: Kali Linux Compilation and Execution

```
(kali@kali)-[~]  
$ nano parallel_resistance.c  
(kali@kali)-[~]  
$ gcc parallel_resistance.c -o parallel_resistance
```

Test Case 1:

```
(kali@kali)-[~]  
$ ./parallel_resistance  
Enter resistance R1 (ohm): 10  
Enter resistance R2 (ohm): 20  
Parallel resistance = 6.67 ohm
```

Test Case 2:

```
(kali@kali)-[~]  
$ ./parallel_resistance  
Enter resistance R1 (ohm): 100  
Enter resistance R2 (ohm): 100  
Parallel resistance = 50.00 ohm
```

Test Case 3:

```
(kali@kali)-[~]  
$ ./parallel_resistance  
Enter resistance R1 (ohm): 4  
Enter resistance R2 (ohm): 6  
Parallel resistance = 2.40 ohm
```